Bloxorz 3D Report

Computer Graphics Course Project

Abstract

Bloxorz 3D is a browser based puzzle game built with Three.js, WebGL, Vite, and modular JavaScript. The player controls a rectangular block on a tile grid and tries to reach the goal tile while avoiding falls and using level mechanics such as fragile tiles, switches, bridges, and teleport split mode.

The main technical idea is to keep the puzzle state exact while making the visual motion smooth. The block always has a clear grid state, but it rolls, falls, splits, merges, and wins through animated 3D transforms. The project also includes a level editor, saved progress, a third-person gameplay camera, procedural audio, particles, postprocessing, and an automated solver check for all official levels.

1. Project goal

Our goal was to build a complete Bloxorz style game that could be demonstrated as a real graphics project, not only as a puzzle logic exercise. The project had to show 3D geometry, transformations, camera control, lighting, shading, interaction, and a working game loop.

The final project includes:

- A playable WebGL game in the browser.
- Eight official levels with verified par values.

- A rolling cuboid with standing and lying orientations.
- Fragile tiles, soft switches, heavy switches, bridges, and teleport split mode.
- A custom level editor.
- Saved progress and best move counts using localStorage.
- Keyboard, touch, and a smooth third-person follow camera.
- Solver based hints for non-teleport levels.
- A validation script that checks all official levels, including the split level.
- Lighting, shadows, physical materials, fog, bloom, vignette, SMAA, particles, and a guarded path tracing integration.

2. System overview

The project is a Vite application. Three.js is used for the scene, camera, geometry, materials, shadows, and rendering. The postprocessing library is used for the render pass, anti-aliasing, bloom, and vignette. A guarded path tracing integration through three-gpu-pathtracer is present, while the presentation build uses the stable rasterized renderer for interaction.

The code is split by responsibility:

- `src/main.js` handles the application state, UI flow, level lifecycle, transitions, hints, path tracing control, and the render loop.
- `src/controls.js` handles keyboard input, touch input, movement rules, undo, falling, switches, teleport splitting, cube movement, and merging.
- `src/grid.js` builds the level tiles, goal, switches, bridges, bridge toggles, and hint highlights.
- `src/block.js` creates the main block and split cubes.
- `src/levels.js` stores the official level data.
- `src/solver.js` contains the BFS hint solver for normal block states and bridge states.
- `scripts/check-levels.js` validates all official levels and includes split mode.
- `src/editor.js` implements custom level editing with raycasting.
- `src/scene.js`, `src/camera.js`, `src/lighting.js`, and `src/themes.js` set up the visual world.
- `src/particles.js`, `src/sounds.js`, and `src/shake.js` add feedback.

The game runs as a single page application. It starts on a home screen, moves into level selection, then into gameplay or the editor. All of these modes share the same WebGL scene, but the active objects and UI layers change.

3. Game flow and user interface

The game has four main user facing states.

- Home screen. It shows the Bloxorz title, a 3D preview scene, and buttons for Play and Editor.
- Level select. It shows eight levels. Locked levels are dimmed. Completed levels are marked, and a star appears when the player reaches par or better.
- Gameplay. It shows the current level name, move counter, key hints, split mode hint when needed, and win or fall overlays.
- Editor. It shows the editor toolbar, tool descriptions, Play, Clear, and Exit buttons.

Progress is saved in localStorage. The game stores completed levels and best move counts. This lets the level select screen show progress after a reload.

The main controls are:

- Arrow keys or WASD to move.
- Z to undo normal block moves.
- R to restart.
- H to request a hint.
- Space to switch active cube in split mode.
- Escape to return to the previous menu state.
- Touch swipes for mobile movement.

The UI is also responsive. The CSS adjusts menus, HUD, hints, and editor controls for smaller screens.

4. Level design

The official game has eight levels. Each level is stored as a two dimensional layout in `src/levels.js`. Tile values represent empty space, normal tiles, goal tiles, fragile tiles, switches, and teleport switches.

The official levels are:

1. First Steps, par 8. This introduces normal movement.
2. Over the Edge, par 7. This introduces narrow paths and falling risk.
3. Fragile Ground, par 6. This introduces fragile tiles.
4. Open Sesame, par 6. This introduces a soft switch and bridge.
5. Heavy Duty, par 12. This introduces a heavy switch.
6. Split Decision, par 8. This introduces teleport split mode.
7. Twin Bridges, par 13. This combines soft and heavy bridge routes.
8. The Gauntlet, par 15. This combines several mechanics in one larger level.

Each level also has a theme. The theme controls sky colors, fog color, tile color, goal color, block color, and sparkle color. This makes each level feel different while keeping the same game rules.

5. Puzzle state

The block state is discrete. This is important because the puzzle must be exact.

The main state is:

```
x, z, orientation
```

The orientation has three possible values:

```
standing
lying_x
lying_z
```

When the block is standing, it occupies one tile. When it lies along the x axis or z axis, it occupies two tiles. The occupied cells are used for every gameplay decision:

- Whether the block is still on the board.
- Whether it is standing on a fragile tile.
- Whether it touched a soft switch.
- Whether it is standing on a heavy switch.
- Whether it is standing on a teleport switch.
- Whether it is standing on the goal.

This separation is one of the cleanest parts of the project. The mesh can move smoothly, but the rule checks remain based on exact tile cells.

6. Movement and animation

Movement begins with an input event. The player presses a key, swipes, or moves a split cube. The code then computes the next state based on the current orientation.

For the full block:

- A standing block rolls into a lying state.
- A lying block can move sideways while staying lying.
- A lying block can roll back into standing if it moves along its long axis.

The visual motion uses two continuous transforms:

- Position interpolation moves the mesh from the old position to the new position.
- Quaternion interpolation rotates the mesh during the roll.

After the animation ends, the block snaps to the exact target position. This avoids small floating point errors from building up over many moves. A small landing squash effect gives the movement a better feel without changing the puzzle state.

Falls have their own animations. If the block is partly hanging over an edge, it pivots around the last valid edge before falling. If it is fully invalid or falls through the goal, it drops downward. These animations use position, rotation, scale, and opacity changes.

7. Tile mechanics

The project has several tile types. They are not only visual. Each one changes validation.

Normal tiles support the block in any legal orientation.

Goal tiles end the level, but only when the full block is standing on the goal. A lying block touching the goal does not win.

Fragile tiles break when the block stands upright on them. The tile falls away, the block sinks slightly, then the block falls.

Soft switches activate when any occupied cell touches them. This means they can be triggered by a standing block, a lying block, or a split cube.

Heavy switches activate only when the full block stands upright on them. This makes the player plan orientation, not just position.

Bridges are hidden or visible tiles controlled by switches. Switch activation toggles bridge targets. The bridge state is stored separately so the solver and undo system can restore it correctly.

Teleport switches split the full block into two cubes. The block disappears and two smaller cubes appear at fixed target cells. The player moves one cube at a time and can switch the active cube with Space. When the two cubes become adjacent, they merge back into a lying block.

8. Editor

The editor lets the user create a custom level inside the same 3D scene. It uses a fixed grid, a top down camera, and raycasting from the mouse position to a grid plane.

The editor tools are:

- Normal tile.
- Fragile tile.
- Goal tile.
- Start tile.
- Soft switch.
- Heavy switch.
- Bridge tile.

- Eraser.

The editor keeps one start and one goal. If the player places a new goal, the old one becomes a normal tile. Start is shown as a marker on top of a normal tile.

Custom bridge behavior is simple by design. All switches in a custom level target all bridge tiles. Bridges start hidden. This keeps the editor easy to use during a short course demonstration.

The player can test the custom level immediately. The game also provides a Back to Editor button after starting a custom level, so the player can edit and test again.

9. Graphics pipeline connection

The project connects directly to the graphics pipeline from the lecture notes.

The level layout begins as data. The code converts that data into geometry. Each tile becomes a rounded box mesh, and the block is another rounded box mesh. The scene also includes particles, sparkles, and UI driven camera transitions.

The model transform places every mesh in world space. Tiles use their row and column to set x and z positions. The block changes position and rotation every time it moves. Split cubes have their own positions.

The view transform comes from the camera. The third-person gameplay camera follows the block from an offset. The menu camera orbits around a small preview level.

The projection step is handled by `THREE.PerspectiveCamera`. It maps the 3D scene into the 2D browser view.

WebGL rasterizes the geometry into fragments. Three.js materials and postprocessing generate shader programs that produce final colors. The frame buffer is the browser canvas shown on screen.

This is the same idea as the lecture pipeline:

```
geometry
model transform
view transform
projection
rasterization
fragment shading
frame buffer
```

10. Transformations and camera

Transformations are used throughout the project.

The block movement uses translation and rotation. The animation updates position and quaternion values across time. When the block falls, the code may rotate it around an edge before applying a downward fall.

The camera also uses transforms. In third-person mode, the camera follows the active block or cube from an offset. It computes a target camera position and a target look-at point, then lerps toward both values so the board stays readable while the block moves.

The camera is not tied to gameplay direction remapping. Movement stays grid-based and always follows the puzzle logic directly.

11. Lighting, materials, and shading

The scene uses several lights:

- Ambient light for base visibility.
- Hemisphere light for soft sky and ground color.
- Directional light as the main sun.
- A small fill light from the opposite side.

The directional light casts shadows. Its target follows the block so shadows stay centered around the active gameplay area.

Tiles and blocks use Three.js physical materials. These materials include parameters such as color, roughness, metalness, clearcoat, emissive color, and environment intensity. Goal tiles pulse with emissive intensity. Switches have small 3D indicators on top.

We did not write custom GLSL for the block. That is worth stating clearly. The project is still shader based because Three.js and the postprocessing library generate WebGL shader programs for materials and effects. The difference is that the shader code is managed by the library.

12. Scene atmosphere and postprocessing

The sky is procedural. The project draws a canvas texture with a vertical color gradient, soft clouds, and simple mountain shapes. The texture becomes the scene background.

Fog is used to soften distance and match the theme color. Each level changes sky, fog, tile, goal, block, and sparkle colors.

The render pipeline also uses postprocessing. SMAA reduces jagged edges, bloom gives bright highlights a soft glow, and vignette adds subtle edge darkening.

Sparkles add small moving light motes near the board. The particle system adds dust when the block lands, shatter particles when a fragile tile breaks, and celebration particles when the player wins.

Sound effects are procedural. They use the Web Audio API, oscillators, noise buffers, gain ramps, and filters. There are sounds for movement, falling, winning, switches, bridge appearance and disappearance, tile break, undo, split, merge, and level start.

Screen shake is used lightly for landing, fragile tile break, and falling. It offsets the camera for a short time and decays quickly.

13. Raster rendering and path tracing integration

The main renderer is real-time rasterization through WebGL and Three.js. This is the renderer used for gameplay, animation, particles, postprocessing, and the live demo.

The project also contains a guarded setup for `three-gpu-pathtracer`. This connects to the ray and path tracing lecture, but it is kept secondary because path tracing is heavier and less suitable for continuous interaction. Rasterization is fast and reliable for the playable game, while the path tracing code shows awareness of the alternative rendering approach and falls back safely if unsupported.

14. Solver and validation

The in-game hint system uses breadth first search for non-teleport levels. BFS explores the move graph level by level, so the first found solution is the shortest solution.

The solver state includes:

```
x, z
orientation
bridge bitmask
move history
```

The bridge bitmask is important. It means the same block position can be a different state if bridge tiles are currently visible or hidden.

The live hint system highlights the target cell or cells for the next move. It also supports hints when the block is lying, not only when it is standing.

Teleport split mode is more complex. The in-game hint solver does not handle teleport switches, but the validation script does. `scripts/check-levels.js` includes an expanded solver that can model block mode, split cube mode, active cube switching, bridge masks, and merging.

The validation command is:

```
npm run check:levels
```

It verifies:

```
Level 1  First Steps       8 moves
Level 2  Over the Edge     7 moves
Level 3  Fragile Ground    6 moves
Level 4  Open Sesame       6 moves
Level 5  Heavy Duty       12 moves
Level 6  Split Decision    8 moves
Level 7  Twin Bridges     13 moves
Level 8  The Gauntlet     15 moves
```

This is useful because it proves the official levels are solvable and that the listed par values match shortest paths.

15. Persistence and undo

Progress is saved in localStorage. The saved data stores completed levels and the best move count per level. This is used by the level select screen to unlock levels and show completed states.

Undo works for normal block mode. Before each normal move, the game saves:

- Block position.
- Block quaternion.
- Block orientation.
- Level layout.
- Bridge states.
- Move count.

When the player presses Z, the game restores that snapshot. Undo is disabled during split mode because split mode changes the structure of the actor from one block into two cubes.

16. Testing and validation

The project was checked in several ways.

`npm run build` confirms the project builds as a production Vite app. The current build passes. Vite reports a large chunk warning because the path tracing dependency is heavy. This is a warning, not a build failure.

`npm run check:levels` verifies all official level par values. The command passes for all eight official levels.

`npm audit` reports zero vulnerabilities after dependency cleanup.

The game was also opened in desktop and mobile sized browser viewports to check that the scene renders, the UI fits, and the responsive CSS does not create horizontal overflow.

17. Limitations and future work

The project is complete enough for the course demonstration, but there are still reasonable future improvements.

- Add split mode support to the live hint system.
- Let the editor choose which bridge tiles each switch controls.
- Add import and export for custom levels.
- Add more official levels.
- Add code splitting so the path tracing dependency does not increase the main bundle size.
- Add optional texture assets for tiles, while keeping the current clean procedural style.
- Add a level preview in the editor before pressing Play.

These are extensions. The current project already includes the full game loop, core mechanics, editor, renderer, validation script, and presentation ready demo flow.

18. Conclusion

Bloxorz 3D combines exact puzzle logic with real time 3D graphics. The game state is discrete, but the visual output uses continuous transforms, camera movement, lighting, materials, shadows, particles, and postprocessing.

The project also includes features that make it feel complete: level progression, saved progress, an editor, a third-person camera, mobile input, procedural sound, and automated level checking.

From a Computer Graphics point of view, the project demonstrates the full path from scene data to final pixels. It uses geometry, model transforms, view and projection, rasterization, fragment shading through Three.js materials, frame buffer output, event driven input, and a guarded path tracing integration for comparison.

Most importantly, it is playable. The player can start at the menu, select a level, solve puzzles, see feedback, unlock progress, build a custom level, and demonstrate the implementation live.